BÀI THỰC HÀNH

LAB 19

Vehicle Speed Monitoring over CAN

Giám sát tốc độ phương tiện qua CAN với EduFramework

S32K144 · MaaZEDU

EduFramework

FPT University

Mục lục

1 Giới thiệu 2
1.1 Tổng quan bài lab 2
1.2 Mục tiêu 2
2 Kiến thức nền 2
2.1 CAN Normal Mode và giao tiếp giữa hai ECU 2
3 Thiết lập phần cứng 3
3.1 Phần cứng sử dụng 3
3.2 Kết nối CAN giữa hai ECU 3
3.3 Cấu hình nguồn và kết nối 4
4 EduFramework API 4
5 Bài thực hành 4
5.1 Yêu cầu 4
5.2 Chương trình 5
5.3 Kiểm chứng 7
6 Mở rộng 7
6.1 Bài tập mở rộng 7
7 Tài liệu tham khảo 7

1 Giới thiệu

1.1 Tổng quan bài lab

Trong hệ thống điện tử ô tô, dữ liệu của một bộ điều khiển điện tử (Electronic Control Unit - ECU) có thể được sử dụng bởi các ECU khác để phục vụ chức năng điều khiển hoặc giám sát. CAN cho phép các bộ điều khiển trao đổi thông điệp trên một đường truyền chung mà không cần kết nối truyền dữ liệu riêng cho từng cặp ECU [2] .
Lab 18 đã giới thiệu Classical CAN và kiểm chứng chức năng truyền nhận bằng Internal Loopback. Lab 19 tiếp tục sử dụng CAN API của EduFramework để xây dựng một hệ thống gồm hai board S32K144 trao đổi dữ liệu qua CAN bus vật lý.
Trong bài thực hành, ECU A giả lập tốc độ phương tiện và phát thông điệp CAN định kỳ. ECU B tiếp nhận dữ liệu, xác định thông điệp phù hợp và hiển thị tốc độ trên Serial Monitor. Hai cửa sổ Serial Monitor được sử dụng để đối chiếu dữ liệu tại phía truyền và phía nhận.

1.2 Mục tiêu

MỤC TIÊU

Các mục tiêu của bài lab bao gồm:

1. Mô tả mô hình truyền thông dữ liệu giữa hai ECU thông qua CAN bus vật lý.
2. Kết nối hai MaaZEDU Development Board và cấu hình CAN ở chế độ Normal.
3. Sử dụng EduFramework để phát và tiếp nhận thông điệp CAN chứa dữ liệu tốc độ.
4. Giải mã dữ liệu nhận được và hiển thị trên Serial Monitor của ECU B.
5. Đối chiếu dữ liệu gửi và nhận để kiểm chứng hoạt động giao tiếp giữa hai ECU.

2 Kiến thức nền

2.1 CAN Normal Mode và giao tiếp giữa hai ECU

Khác với Internal Loopback, chế độ Normal sử dụng đường truyền CAN bên ngoài. Một CAN controller tạo và xử lý frame ở mức giao thức; CAN transceiver trên board kết nối bộ điều khiển với cặp tín hiệu vi sai `CAN_H` và `CAN_L` của bus [2] .
Để hai ECU trao đổi thông điệp, các nút cần kết nối đúng đường CAN, sử dụng bitrate tương thích và có cấu hình kết thúc bus phù hợp. Ví dụ FlexCAN trên S32K144 có thể hoạt động ở `500 kbit/s` [1] .

Đặc điểm	Lab 18	Lab 19
Chế độ CAN	Internal Loopback	Normal
Số ECU	Một board	Hai board
Đường truyền	Nội bộ CAN controller	CAN bus vật lý
Mục đích	Kiểm chứng truyền nhận nội bộ	Trao đổi dữ liệu giữa hai ECU

Bảng 19.2.1. So sánh phạm vi thực hành CAN ở Lab 18 và Lab 19

Trong EduFramework, `CAN_begin()` khởi tạo CAN ở chế độ Normal theo mặc định. Do đó, Lab 19 không cần gọi `CAN_setMode()` để chuyển chế độ. Các chi tiết về cấu trúc Classical CAN frame, arbitration và cơ chế kiểm tra lỗi đã được trình bày ở Lab 18 [4] .

3 Thiết lập phần cứng

3.1 Phần cứng sử dụng

Bài thực hành sử dụng hai MaaZEDU Development Board, một board làm ECU A và board còn lại làm ECU B. Mỗi board cần nguồn ngoài và kết nối USB riêng để nạp chương trình và theo dõi Serial Monitor.

Phần cứng	Mô tả
MaaZEDU Development Board (2)	Hai board S32K144 đóng vai trò ECU A và ECU B.
12V DC Adapter (2)	Cấp nguồn ngoài cho hai board MaaZEDU.
USB Cable (2)	Nạp chương trình và sử dụng Serial Monitor riêng cho mỗi ECU.
Jumper Wires (3)	Kết nối `CAN_H` , `CAN_L` và GND giữa hai board.

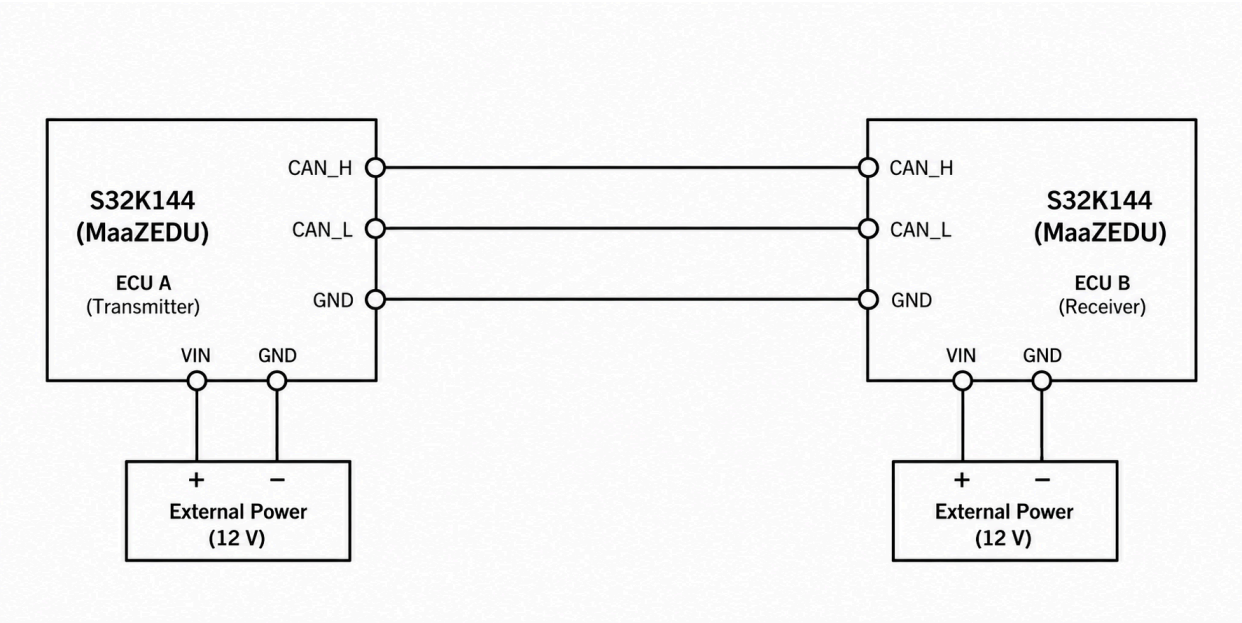
Bảng 19.3.1. Phần cứng sử dụng trong Lab 19

3.2 Kết nối CAN giữa hai ECU

Nối hai đầu giao tiếp CAN có sẵn trên MaaZEDU theo bảng dưới đây. Các tín hiệu `CAN_H` và `CAN_L` là đường truyền của CAN bus, không phải chân `CAN0_TX` và `CAN0_RX` trực tiếp trên MCU [5] .

ECU A	ECU B	Vai trò
`CAN_H`	`CAN_H`	Đường tín hiệu CAN High.
`CAN_L`	`CAN_L`	Đường tín hiệu CAN Low.
`GND`	`GND`	Tham chiếu điện thế chung giữa hai ECU.

Bảng 19.3.2. Kết nối CAN bus giữa ECU A và ECU B



Hình 19.3.1. Sơ đồ kết nối hai ECU qua CAN bus trên MaaZEDU

3.3 Cấu hình nguồn và kết nối

Chuyển jumper chọn nguồn trên cả hai board sang vị trí nguồn ngoài trước khi cấp adapter `12V` theo cấu hình của MaaZEDU. Giữ kết nối USB với máy tính để nạp chương trình và quan sát Serial Monitor [5] .
Kiểm tra điện trở kết thúc CAN bus tại hai đầu đường truyền theo cấu hình thực tế của board. Không bổ sung điện trở nếu các đầu bus đã được kết thúc phù hợp [2] .

Ghi chú

Tắt nguồn trước khi thay đổi jumper hoặc đấu nối. Không nối tắt `CAN_H` với `CAN_L` và không cấp `12V` trực tiếp vào các chân tín hiệu. Hai ECU phải sử dụng cùng bitrate `500 kbit/s` trong bài thực hành.

4 EduFramework API

Lab 19 sử dụng lại các CAN API đã giới thiệu trong Lab 18. Mỗi ECU đảm nhận một phần công việc: ECU A tạo và gửi frame, ECU B kiểm tra hàng đợi nhận và đọc frame đã nhận. Ứng dụng sử dụng `Serial1` để hiển thị kết quả [4] .

API / Thành phần	Vai trò trong Lab 19
`CAN_begin()`	Khởi tạo FlexCAN0 ở `500 kbit/s` , chế độ Normal.
`CAN_Frame_t`	Biểu diễn CAN frame gồm `id` , `format` , `length` và `data[]` .
`CAN_STANDARD`	Chọn định dạng CAN identifier 11-bit.
`CAN_send()`	ECU A gửi một CAN frame theo cơ chế blocking.
`CAN_available()`	ECU B kiểm tra hàng đợi có frame sẵn sàng để đọc.
`CAN_read()`	ECU B lấy frame nhận được theo cơ chế non-blocking.

Bảng 19.4.1. Các CAN API và thành phần được sử dụng trong Lab 19

Trong chương trình, `CAN_send()` trả về `true` khi thao tác truyền hoàn thành thành công. Phía nhận chỉ xử lý dữ liệu khi `CAN_available()` và `CAN_read()` đều cho kết quả hợp lệ. Cấu trúc tham số và giá trị trả về của từng API được trình bày chi tiết ở Lab 18.

5 Bài thực hành

5.1 Yêu cầu

Xây dựng ứng dụng truyền thông CAN giữa hai ECU trên một bus vật lý. ECU A mô phỏng tốc độ phương tiện trong khoảng `0..200 km/h` , thay đổi từng bước `10 km/h` và truyền thông điệp mỗi `500 ms` . ECU B nhận thông điệp, giải mã và in tốc độ nhận được trên Serial Monitor.
Cả hai ECU sử dụng Standard CAN ID `0x100` , bitrate `500 kbit/s` và frame chứa `3 byte` payload [3] . Byte đầu chứa số thứ tự thông điệp (`SEQ`); hai byte còn lại chứa giá trị tốc độ. Đây là định dạng dữ liệu do bài lab quy ước, không phải định dạng tốc độ xe chuẩn hóa cho mọi phương tiện.

Hai vai trò sử dụng chung file `src/main.c`, được lựa chọn bằng cách thay đổi `#define ECU 0` (truyền) hoặc `#define ECU 1` (nhận). `Serial1` hoạt động ở `9600 baud` trên cả hai board.

5.2 Chương trình

```
#include "Arduino.h"
#include "can.h"
#define ECU 0  // 0: Transmitter, 1: Receiver
#define CAN_BITRATE 500000UL
#define CAN_ID      0x100UL

int main(void)
{
    CAN_Frame_t txFrame = {0};
    CAN_Frame_t rxFrame = {0};
    uint16_t speed = 0U;
    uint8_t sequence = 0U;
    bool increasing = true;
    setup();
    Serial1_begin(9600U);
    if (false == CAN_begin(CAN_BITRATE))
    {
        Serial1_println("CAN initialization failed.");
        while (1) {}
    }
#if (ECU == 0)
    Serial1_println("=== ECU A: Transmitter ===");
    txFrame.id = CAN_ID;
    txFrame.format = CAN_STANDARD;
    txFrame.length = 3U;
    while (1)
    {
        txFrame.data[0] = sequence;
        txFrame.data[1] = (uint8_t)(speed & 0xFFU);
        txFrame.data[2] = (uint8_t)(speed >> 8U);
        if (true == CAN_send(&txFrame))
        {
            Serial1_print("TX | SEQ: ");
            Serial1_printInt(sequence);
            Serial1_print(" | Speed: ");
            Serial1_printInt(speed);
            Serial1_println(" km/h");
        }
        else
        {
            Serial1_println("CAN transmission failed.");
        }
```

Mã nguồn 19.5.1. Chương trình Lab 19 - phần khởi tạo và truyền CAN của ECU A (1/2)

```
        sequence++;
        if (true == increasing)
        {
            if (speed < 200U) { speed += 10U; }
            else { increasing = false; speed -= 10U; }
        }
        else
        {
            if (speed > 0U) { speed -= 10U; }
            else { increasing = true; speed += 10U; }
        }
        delay(500U);
    }
#elif (ECU == 1)
    Serial1_println("=== ECU B: Receiver ===");
    while (1)
    {
        if (true == CAN_available())
        {
            if (true == CAN_read(&rxFrame))
            {
                if ((CAN_ID == rxFrame.id) &&
                    (CAN_STANDARD == rxFrame.format) &&
                    (3U == rxFrame.length))
                {
                    sequence = rxFrame.data[0];
                    speed = (uint16_t)rxFrame.data[1] |
                            ((uint16_t)rxFrame.data[2] << 8U);
                    Serial1_print("RX | SEQ: ");
                    Serial1_printInt(sequence);
                    Serial1_print(" | Speed: ");
                    Serial1_printInt(speed);
                    Serial1_println(" km/h");
                }
            }
        }
    }
#else
#error "Invalid ECU selection"
#endif
}
```

Mã nguồn 19.5.2. Chương trình Lab 19 - cập nhật tốc độ và nhận CAN tại ECU B (2/2)

Trong phần khởi tạo, `setup()` chuẩn bị môi trường EduFramework, `Serial1_begin(9600U)` mở giao diện giám sát và `CAN_begin(500000UL)` khởi tạo CAN ở chế độ Normal. Nếu khởi tạo thất bại, chương trình dừng trước khi truy cập bus.

Khi `ECU` bằng `0`, ứng dụng cấu hình frame với ID `0x100` và ba byte dữ liệu. `data[0]` mang `sequence`, `data[1]` chứa tám bit thấp và `data[2]` chứa tám bit cao của biến `speed`. Sau khi gửi, chương trình hiển thị thông tin TX, cập nhật biến tốc độ trong giới hạn `0..200 km/h` và chờ `500 ms` trước chu kỳ tiếp theo.

Khi `ECU` bằng `1`, chương trình chờ frame trong hàng đợi nhận. Chỉ những frame có ID `0x100`, định dạng Standard và độ dài `3` được xử lý. ECU B lấy lại giá trị tốc độ từ hai byte dữ liệu bằng phép OR và dịch trái tám bit, sau đó hiển thị `SEQ` và `Speed`. `SEQ` được in để đối chiếu các frame tương ứng giữa hai Serial Monitor; chương trình chưa tự động đánh giá tình trạng mất frame.

5.3 Kiểm chứng

Đặt `#define ECU 0`, build và nạp chương trình cho ECU A. Sau đó đổi thành `#define ECU 1` và nạp cho ECU B. Kết nối hai board theo sơ đồ, cấp nguồn ngoài `12V` và mở hai Serial Monitor ở `9600 baud`. Quan sát dữ liệu hiển thị ở mỗi ECU.
Kết quả đã kiểm chứng trên ECU A:

```
=== ECU A: Transmitter ===
TX | SEQ: 0 | Speed: 0 km/h
TX | SEQ: 1 | Speed: 10 km/h
TX | SEQ: 2 | Speed: 20 km/h
TX | SEQ: 3 | Speed: 30 km/h
```

Kết quả đã kiểm chứng trên ECU B:

```
=== ECU B: Receiver ===
RX | SEQ: 0 | Speed: 0 km/h
RX | SEQ: 1 | Speed: 10 km/h
RX | SEQ: 2 | Speed: 20 km/h
RX | SEQ: 3 | Speed: 30 km/h
```

Đối chiếu các frame có cùng `SEQ` giữa hai cửa sổ. Trong các mẫu đã kiểm thử, ECU B nhận chính xác các giá trị tốc độ tương ứng với ECU A. Có thể quan sát thêm khi tốc độ tăng đến `200 km/h` rồi giảm về `0 km/h` để kiểm chứng đầy đủ chu kỳ mô phỏng.

KẾT QUẢ MONG ĐỢI

Hai ECU khởi tạo CAN ở `500 kbit/s` và trao đổi thông điệp trên CAN bus vật lý. ECU A phát dữ liệu tốc độ định kỳ; ECU B tiếp nhận frame có ID và định dạng phù hợp, giải mã và hiển thị đúng giá trị tốc độ của các thông điệp đã nhận. Thông tin `SEQ` cho phép đối chiếu dữ liệu TX/RX trên hai Serial Monitor.

6 Mở rộng

6.1 Bài tập mở rộng

Bổ sung chức năng **Vehicle Overspeed Warning** cho ECU B. Khi tốc độ nhận được lớn hơn `100 km/h`, Serial Monitor hiển thị thêm thông báo `WARNING: Overspeed!`. Khi tốc độ không vượt ngưỡng, chương trình tiếp tục hiển thị dữ liệu như bài thực hành chính.
Chỉ thay đổi cách xử lý dữ liệu sau khi ECU B nhận và giải mã frame; giữ nguyên CAN ID, định dạng payload và chu kỳ phát của ECU A. Bài tập không yêu cầu cảm biến hoặc thiết bị bổ sung.

7 Tài liệu tham khảo

[1] NXP Semiconductors, *S32K1xx Series Cookbook* , AN5413, Rev. 5, 2020.

[2] Steve Corrigan, Texas Instruments, *Introduction to the Controller Area Network (CAN)* , SLOA101B, Rev. B, 2016.

[3] CAN in Automation (CiA), *CAN CC (Classical CAN)* , CiA CAN Knowledge.

[4] EduFramework, *Arduino-style CAN API (can.h, can.c)* , EduFramework Source Code.

[5] MaaZEDU, *MaaZEDU Development Board Guide* , Rev. 1.0.